

Lottoziehung in Python

6 aus 45 – Ziehung mit Slicing und Statistik

Überblick: Was verlangt die Aufgabe?

Anforderung aus der Aufgabe	Wo im Code gelöst
Lottoziehung als Methode mit dem Zufallsgenerator	<code>lottoziehung()</code>
Keine Zahl darf zweimal gezogen werden	Gezogene Zahl wird per Slicing ans Listenende geschoben
Für alle 45 Zahlen nur 45 Zufallsaufrufe	Zufallsbereich wird mit <code>44 - k</code> jedes Mal kleiner
Sechs Zahlen ziehen und ausgeben	Testteil, <code>print(...)</code>
Statistik als Methode, 1000 / 10000 / 100000 Ziehungen	<code>lotto_statistik(n)</code>
Dictionary: wie oft wurde welche Zahl gezogen	<code>statistik = {}</code> mit Zählern 0
Statistikmethode nach jeder Ziehung aufrufen, Zähler erhöhen	<code>statistik_aktualisieren()</code>

Hinweis: Eine Funktion `random.getrand()` gibt es in Python nicht. Gemeint ist das `random`-Modul. Wir verwenden `random.randint(a, b)`, das eine ganze Zufallszahl zwischen `a` und `b` liefert (beide Grenzen inklusive).

Schritt 0: Import

```
import random
```

`import random` lädt Pythons eingebautes Modul für Zufallszahlen. Erst danach kann man `random.randint()` benutzen.

Teil 1: Die Lottoziehung als Methode

```
def lottoziehung(anzahl=6):
    zahlen = list(range(1, 46))
    gezogene_zahlen = []

    for k in range(anzahl):
        letzter_index = 44 - k
        zufalls_index = random.randint(0, letzter_index)

        gezogene = zahlen[zufalls_index]
        gezogene_zahlen.append(gezogene)

        zahlen = zahlen[:zufalls_index] + zahlen[zufalls_index+1:] + [gezogene]

    return gezogene_zahlen
```

1.1 Der Kopf der Methode

`def lottoziehung(anzahl=6)` definiert die Methode. Der Parameter hat den Standardwert 6, also liefert `lottoziehung()` sechs Zahlen. Mit `lottoziehung(45)` kann man aber auch alle Zahlen ziehen – das brauchen wir später als Beweis, dass keine Zahl doppelt kommt.

1.2 Die Liste mit allen Zahlen

`list(range(1, 46))` erzeugt `[1, 2, 3, ..., 45]`. Das ist die Lottotrommel mit allen Kugeln. `range` schließt die obere Grenze aus, darum steht dort 46 und nicht 45. Am Anfang liegt an Stelle 0 die Zahl 1, an Stelle 44 die Zahl 45, weil Python bei 0 zu zählen beginnt. `gezogene_zahlen` ist eine leere Liste für die Gewinnzahlen.

1.3 Die Grundidee: Liste in zwei Bereiche teilen

Die Liste hat immer 45 Einträge, sie wird nie kürzer. Stattdessen denken wir sie uns zweigeteilt: **vorne** liegen die Zahlen, die noch gezogen werden können, **hinten** sammeln sich die schon gezogenen. Jede gezogene Zahl wird ganz ans Ende geschoben, und der Zufall darf ab dann nur noch im vorderen Bereich wählen.

Durchgang k	letzter_index	Zufall wählt aus	Bereich hinten (schon gezogen)
0	44	Stellen 0 bis 44 (45 Zahlen)	leer
1	43	Stellen 0 bis 43 (44 Zahlen)	Stelle 44
2	42	Stellen 0 bis 42 (43 Zahlen)	Stellen 43 bis 44
5	39	Stellen 0 bis 39 (40 Zahlen)	Stellen 40 bis 44

1.4 Den zufälligen Index wählen

`letzter_index = 44 - k` berechnet, wo der vordere Bereich endet. Im ersten Durchgang ist `k = 0`, also geht der Bereich bis 44. Im zweiten bis 43, und so weiter. `random.randint(0, letzter_index)` würfelt dann eine Stelle nur aus diesem Bereich. Weil hinten die schon gezogenen Zahlen liegen und der Zufall dort nie hinkommt, kann keine Zahl doppelt gezogen werden.

Wichtig: Wir würfeln eine **Stelle**, keine Zahl. Mit `zahlen[zufalls_index]` holen wir uns dann die Zahl, die an dieser Stelle liegt, und hängen sie an `gezogene_zahlen` an. Man darf nicht einfach `zufalls_index + 1` rechnen, denn nach dem Verschieben stehen die Zahlen nicht mehr in der ursprünglichen Reihenfolge.

1.5 Das Slicing: die gezogene Zahl nach hinten schieben

```
zahlen = zahlen[:zufalls_index] + zahlen[zufalls_index+1:] + [gezogene]
```

Diese Zeile baut eine neue Liste aus drei Teilen zusammen:

Teil	Bedeutung
<code>zahlen[:zufalls_index]</code>	alle Zahlen vor der gezogenen (von Anfang bis ausschließlich Index)
<code>zahlen[zufalls_index+1:]</code>	alle Zahlen nach der gezogenen (ab Index + 1 bis zum Ende)
<code>[gezogene]</code>	die gezogene Zahl selbst, als Liste mit einem Element

Das `+` hängt Listen aneinander. Die gezogene Zahl fehlt in den ersten beiden Teilen und landet durch den dritten Teil ganz hinten. Die eckigen Klammern um `gezogene` sind nötig, weil man nur eine Liste an eine Liste hängen kann, keine einzelne Zahl. Die Liste bleibt so immer 45 Einträge lang.

Merke: Beim Slicing ist der Anfang inklusive, das Ende exklusive. `zahlen[:3]` sind die Stellen 0, 1, 2. `zahlen[4:]` ist alles ab Stelle 4. Zusammen fehlt genau Stelle 3 – das ist der Trick.

1.6 Ein Beispiel Schritt für Schritt

Mit einer kleinen Liste aus 6 Zahlen (dann wäre `letzter_index = 5 - k`). Der fett gedruckte Teil hinten sind die schon gezogenen Zahlen:

k	Bereich	Index	Gezogen	Liste nachher
0	0 bis 5	1	2	[1, 3, 4, 5, 6, 2]
1	0 bis 4	3	5	[1, 3, 4, 6, 2, 5]
2	0 bis 3	0	1	[3, 4, 6, 2, 5, 1]
3	0 bis 2	2	6	[3, 4, 2, 5, 1, 6]

Im zweiten Durchgang zum Beispiel: Die Liste ist [1, 3, 4, 5, 6, 2], Index 3 wird gewürfelt, dort liegt die 5. `zahlen[:3] = [1, 3, 4]`, `zahlen[4:] = [6, 2]`, dazu [5] → [1, 3, 4, 6, 2, 5]. Die 2 von vorhin liegt immer noch hinten und ist für den Zufall unerreichbar, weil der Bereich nur noch bis Stelle 4 geht.

1.7 Beweis: 45 Zahlen mit genau 45 Aufrufen

Die Schleife `for k in range(anzahl)` läuft genau `anzahl`-mal, und in jedem Durchgang steht genau ein `randint`. Bei 45 Zahlen sind das 45 Aufrufe, und jeder trifft eine neue Zahl. Im letzten Durchgang ist `letzter_index = 0`, dann bleibt nur noch eine Zahl übrig. Das Programm prüft

das auch:

```
alle = lottoziehung(45)
print("Alle 45 gezogen, keine doppelt:", len(set(alle)) == 45)
```

Ein set speichert jeden Wert nur einmal. Hat es 45 Elemente, waren alle Zahlen verschieden. Ich habe die randint-Aufrufe zusätzlich mitgezählt: genau 45.

Teil 2: Die Statistik als Methode

```
def statistik_aktualisieren(statistik, ziehung):
    for zahl in ziehung:
        statistik[zahl] = statistik[zahl] + 1

def lotto_statistik(anzahl_ziehungen):
    statistik = {}
    for z in range(1, 46):
        statistik[z] = 0

    for a in range(anzahl_ziehungen):
        ziehung = lottoziehung()
        statistik_aktualisieren(statistik, ziehung)

    return statistik
```

2.1 Das Dictionary erstellen

Ein Dictionary speichert Paare aus **Schlüssel** und **Wert**. Hier ist der Schlüssel die Lottozahl und der Wert, wie oft sie gezogen wurde. `statistik = {}` legt ein leeres Dictionary an, die Schleife trägt dann jede Zahl von 1 bis 45 mit dem Zähler 0 ein. Ergebnis: {1: 0, 2: 0, ..., 45: 0}.

2.2 Die Statistikmethode: Zähler erhöhen

`statistik_aktualisieren` bekommt das Dictionary und die sechs Zahlen einer Ziehung. Für jede Zahl wird ihr Zähler um eins erhöht: `statistik[zahl] = statistik[zahl] + 1` liest den alten Wert, rechnet plus 1 und speichert ihn wieder.

Die Methode braucht kein `return`. Ein Dictionary ist veränderbar, und die Methode bekommt keine Kopie, sondern dasselbe Dictionary. Die Änderungen sieht man deshalb auch in `lotto_statistik`.

Ziehung	Gezogene Zahlen	Zähler danach (Auszug)
Start	-	{3: 0, 17: 0, 22: 0, ...}
1	[3, 17, 22, 30, 41, 44]	{3: 1, 17: 1, 22: 1, ...}
2	[5, 17, 22, 28, 33, 40]	{3: 1, 17: 2, 22: 2, ...}
3	[3, 9, 12, 17, 35, 45]	{3: 2, 17: 3, 22: 2, ...}

2.3 Viele Ziehungen durchführen

lotto_statistik legt zuerst das Dictionary an, zieht dann so oft wie gewünscht und ruft, wie in der Aufgabe verlangt, **nach jeder einzelnen Ziehung** die Statistikmethode auf. Am Ende kommt das fertige Dictionary zurück. Die Variablen a und z in den Schleifen sind nur Zähler.

Warum eine eigene Methode?: Die Aufgabe sagt "Ruf die Statistikmethode nach jeder Ziehung auf". Darum steht das Hochzählen in einer eigenen Methode statistik_aktualisieren. Würde man die Schleife direkt in lotto_statistik schreiben, käme das gleiche Ergebnis heraus, aber es gäbe keine eigene Statistikmethode, die aufgerufen wird.

Teil 3: Testen des Programms

```
print("Gezogene Lottozahlen:", lottoziehung())

alle = lottoziehung(45)
print("Alle 45 gezogen, keine doppelt:", len(set(alle)) == 45)

for durchlaeufer in [1000, 10000, 100000]:
    ergebnis = lotto_statistik(durchlaeufer)

    print(f"\nStatistik nach {durchlaeufer} Ziehungen:")
    for zahl in ergebnis:
        print("Zahl", zahl, "wurde", ergebnis[zahl], "mal gezogen")
```

Zuerst wird eine Ziehung ausgegeben, dann der 45er-Beweis. Die Schleife über [1000, 10000, 100000] setzt das "usw." aus der Aufgabe um. Wer mehr will, hängt 1000000 an die Liste. Das f vor dem Text ist ein f-String: Variablen in geschweiften Klammern werden direkt eingesetzt. \n macht eine Leerzeile. for zahl in ergebnis geht alle Schlüssel des Dictionaries durch, ergebnis[zahl] liefert die Anzahl dazu.

Beispielausgabe (gekürzt):

```
Gezogene Lottozahlen: [20, 27, 36, 18, 33, 8]
Alle 45 gezogen, keine doppelt: True

Statistik nach 1000 Ziehungen:
Zahl 1 wurde 148 mal gezogen
Zahl 2 wurde 131 mal gezogen
Zahl 3 wurde 108 mal gezogen
...
```

Was zeigt die Statistik?

Pro Ziehung kommen 6 von 45 Zahlen, also hat jede Zahl die Chance $6/45 = 13,3\%$. Bei 1000 Ziehungen sollte jede Zahl etwa $1000 \cdot 6 / 45 = 133$ -mal vorkommen. Ein Testlauf ergab:

Ziehungen	Erwartet pro Zahl	Seltenste Zahl	Häufigste Zahl	Größte Abweichung
1.000	133	116-mal	157-mal	17,7 %
10.000	1.333	1.248-mal	1.402-mal	6,4 %
100.000	13.333	13.141-mal	13.499-mal	1,4 %

Bei wenigen Ziehungen schwanken die Werte stark. Je mehr Ziehungen, desto näher liegen alle Zahlen am Erwartungswert. Das ist das **Gesetz der großen Zahlen**, und es zeigt, dass der Algorithmus fair ist. Deine Werte sehen bei jedem Lauf etwas anders aus.

Typischer Denkfehler: Eine Zahl, die bisher selten kam, hat bei der nächsten Ziehung keine höhere Chance. Jede Ziehung startet mit einer frischen Liste `list(range(1, 46))`.

Der komplette Code

```
import random
def lottoziehung(anzahl=6):
    # Liste mit den Zahlen 1 bis 45 erstellen
    zahlen = list(range(1, 46))
    gezogene_zahlen = []
    # 6 Zahlen ziehen
    for k in range(anzahl):
        # Das aktuelle Ende der Liste (wird mit jedem Durchgang 1 kleiner)
        letzter_index = 44 - k
        # Einen zufälligen Index im gültigen Bereich wählen
        zufalls_index = random.randint(0, letzter_index)
        # Zahl am Index merken
        gezogene = zahlen[zufalls_index]
        gezogene_zahlen.append(gezogene)
        # PER SLICING: Die gezogene Zahl ans Ende verschieben!
        # Wir nehmen alle Zahlen vor und nach dem Index und hängen die gezogene Zahl ganz
        # hinten an
        zahlen = zahlen[:zufalls_index] + zahlen[zufalls_index+1:] + [gezogene]
    return gezogene_zahlen
def statistik_aktualisieren(statistik, ziehung):
    # Wird nach JEDER Ziehung aufgerufen: Zähler jeder gezogenen Zahl um 1 erhöhen
    for zahl in ziehung:
        statistik[zahl] = statistik[zahl] + 1
def lotto_statistik(anzahl_ziehungen):
    # Dictionary für die Statistik erstellen (Zahl 1-45 hat am Anfang 0 Ziehungen)
    statistik = {}
    for z in range(1, 46):
        statistik[z] = 0
    # Mehrere Ziehungen durchführen
    for a in range(anzahl_ziehungen):
        ziehung = lottoziehung()
        statistik_aktualisieren(statistik, ziehung)
    return statistik
# --- Testen des Programms ---
# Eine einfache Ziehung
print("Gezogene Lottozahlen:", lottoziehung())
# Beweis: alle 45 Zahlen ziehen -> keine doppelt
alle = lottoziehung(45)
print("Alle 45 gezogen, keine doppelt:", len(set(alle)) == 45)
# Statistik mit 1000, 10000 und 100000 Ziehungen
for durchlaeufer in [1000, 10000, 100000]:
    ergebnis = lotto_statistik(durchlaeufer)
    print(f"\nStatistik nach {durchlaeufer} Ziehungen:")
    for zahl in ergebnis:
        print("Zahl", zahl, "wurde", ergebnis[zahl], "mal gezogen")
```